COMP 2402 AB - Fall 2021
Assignment #5

Due: FRIDAY December 10, 23:59

Submit early and often.
PLEASE REMOVE ALL PRINT STATEMENTS THAT ARE NOT NEEDED.
No late submissions will be accepted.

Academic Integrity

You may:

- Discuss general approaches with course staff,
- Discuss general approaches with your classmates,
- Use code and/or ideas from the textbook,
- Use a search engine / the internet to look up basic Java syntax,
- Send your code / screen share your code with course staff (although it is unlikely the course staff has the bandwidth to look at individuals' code, unfortunately.)

You may not:

- Send or otherwise share code or code snippets with classmates,
- Use code not written by you, unless it is code from the textbook (and you should cite it in comments),
- Use a search engine / the internet to look up approaches to the assignment,
- Use code from previous iterations of the course, unless it was solely written by you,
- Use the internet to find source code or videos that give solutions to the assignment.

If you ever have any questions about what is or is not allowable regarding academic integrity, please do not hesitate to reach out to course staff. We will be happy to answer. Sometimes it is difficult to determine the exact line, but if you cross it the punishment is severe and out of our hands. Any student caught violating academic integrity, whether intentionally or not, will be reported to the Associate Dean and be penalized as follows:

- First offence: F in the course.
- Second offence: One-year suspension from program.
- Third offence: Expulsion from the University.

These are standard penalties. More-severe penalties will be applied in cases of egregious offences. For more information, please see Carleton University's Academic Integrity page.

Grading

This assignment will be tested and graded by a computer program (and **you can submit as many times as you like, your highest grade is recorded**). For this to work, there are some important rules you must follow:

- Keep the directory structure of the provided zip file. **If you find a file in the subdirectory comp2402a5 leave it there.**
- Keep the package structure of the provided zip file. **If you find a package comp2402a5; directive at the top of a file, leave it there.**
- Do not rename or change the visibility of any methods already present. If a method or class is public leave it that way.
- Do not change the main(String) method of any class. Some of these are setup to read command line arguments and/or open input and output files. Don't change this behaviour. Instead, learn how to use command-line arguments to do your own testing.
- **Submit early and often.** The submission server compiles and runs your code, and gives you a mark. You can submit as often as you like and only your latest submission will count. There is no excuse for submitting code that does not compile or does not pass tests.
- Write efficient code. The submission server places a limit on how much time it will spend executing your code, even on inputs with a million lines. For some questions it also places a limit on how much memory your code can use. If you choose and use your data structures correctly, your code will easily execute within the time limit. Choose the wrong data structure, or use it the wrong way, and your code will be too slow for the submission server to grade (resulting in a grade of 0).

Submitting and Server Tests

The submission server is available. If you have any issues, please post on piazza with a clear description of the problem (the text of the error message) and we'll help you as soon as we can. As the deadline approaches, we may have to make changes to the server (see changes described below), but I'm leaving it as-is for now, while its performance holds up.

When you submit your code, the server runs tests on your code similar to how the local tests work. These are bigger and better tests than the small number of tests provided in the "Local Tests" section further down this document. They are obfuscated from you, because you should try to find exhaustive tests of your own code. This can be frustrating because you are still learning to think critically about your own code, to figure out where its weaknesses are. But have patience with yourself and make sure you read the question carefully to understand its edge cases.

Warning: Do not wait until the last minute to submit your assignment. There is a hard 5 second limit on the time each test has to complete. If the server is heavily loaded, borderline tests may start to fail. **You can submit multiple times and your best score is recorded.**

Changes from Assignment 4's Server:

The server for this assignment should be much improved over the last assignments.

- Some tests may have specific "prerequisite" tests that must pass before they are run. For example, if you fail an early time performance test, the server may not even bother trying the longer time performance test. This should make the server *so* much faster.
- You have to pass all the tests on a given part to get full points for that part, even if some of the tests (such as correctness tests) are worth 0. This way you can't get full points for an incorrect-on-small-cases-but-correct-on-large-performance submission.

The Assignment

Purpose & Goals

Generally, the assignments in this course are meant to give you the opportunity to practice with the topics of this course in a way that is challenging yet also manageable. At times you may struggle and at others it may seem more straight-forward; just remember to keep trying and practicing, and over time you will improve.

Specifically, assignment 5 aims to improve your understanding of the hashtable- and tree-based data structures we've been talking about by:

- Encouraging you to think about how you need to use and access data before choosing the interface / data structure to store the data, with a focus on thinking about which of a List, SSet, PriorityQueue, USet/HashSet, Dictionary/Hashtable or Graph (provided in starter code) is more appropriate, in a variety of problems, and
- Implementing the `SubsetMeld` interface, using some of the concepts we've been learning about.

Note that **since there is no Graph interface in java, one has been provided for you**. You shouldn't have to change anything in the Graph, AdjacencyLists, or Algorithms classes, although you may do so if you choose.

Part A: The Setup

Start by downloading and decompressing the Assignment 5 Zip File (comp2402a5.zip), which contains a skeleton of the code you need to write. You should have the files: Part1.java, Part2.java, Part3.java, Part4.java, Graph.java, AdjacencyLists.java, Algorithms.java, SubsetMeld.java, SlowSubsetMeld.java, MySubsetMeld.java and Tester.java.

As with assignments 1-4, you can test Part B using the command line or input/output files. You can test Part C with the Tester class included in the zip file. **As before, you'll want to add your own tests as this is just given as a basis on which you should build.** The submission server will do thorough testing; it is your job to read the specifications as carefully as possible, and to examine your own code for potential problems and test those cases.

If you are having trouble running these programs, **figure this out first before attempting to do the assignment**. Check the assignment 5 FAQ on discord and ask a question there if you are stuck and course staff or another student will likely help you fairly quickly.

Part B: The Interface

As with assignments 1-4 you will be filling in the `doIt` method in each of the appropriate `PartX` classes. You can go back to assignments 1-4 as reference, and you may use the sample solutions as a starting point if you like.

You can download some sample input and output files for each question as a zip file (a5-io.zip). If you find that a particular question is unclear, you can possibly clarify it by looking at the sample files for that question. **Note that these are very limited tests; you can and should add to them to test your code more thoroughly** as the server tests are much more extensive.

Note: Java does not have a Graph interface, so one is provided for you in the starter files, along with an AdjacencyLists implementation and some common Graph algorithms in the Algorithms class. You can see examples of how to construct a graph within those files, if that is what you want to do.

1. [5 marks] Read the input one line at a time, and output a line the first time it is seen. For example, the input "a", "c", "a", "b", "c", "a" would output "a", "c", "b".

2. [5 marks] Read the input one line at a time, and output a line the `x`'th time it is seen, where `x>=1` is a parameter to the `doIt` method. For example, when x=3 and the input is "a", "c", "a", "b", "c", "**c**", "**a**", "a" the output is "c", "a".

3. [10 marks] Read the input one line at a time and output "yes" if there is a pair of (not necessarily consecutive) lines whose lengths sum to x, and "no" otherwise, where `x>=0` is a parameter to the `doIt` method. For example, on input "abcde", "fgh", "ijklmno", the output would be "yes" for x=8 or x=10, and "no" for x=6 or x=7.

4. [20 marks] Consider the following 2-player "road trip" game: player 1 starts with a word w1 (typically country or city); player 2 then names a new word w2 that starts with the last letter of w1. Player 1 then names a new word w3 that starts with the last letter of w2, and so on. For example, this game might play out as "dortmun**d**", "**d**ominican republi**c**",

"**c**anad**a**", "**a**jax". Read in the input one line at a time, and output the fewest number of turns it might take for such a game that starts at line 1, ends at line n (where n is the total number of lines in the file), using only lines of the file as the possible words in between. If there is no way to play to get to the last line from the first line, output 0. For example, the input "dortmund", "bermuda", "canada", "paris", "dominican republic", "ajax" would return 4 ("dortmund" -> "dominican republic" -> "canada" -> "ajax"), whereas the input "dortmund", "bermuda", "canada", "dominican republic", "ajax", "paris" would return 0 since there is no way of getting from "dortmund" to "paris" on just the input lines. Note that there may be many "ways" of playing to get from the first line to the last line, and you want to return the fewest number of turns that do so. Each turn must use a line that is not used on any previous turn. You may assume that each line has at least one character on it, and case matters (so, you do not need to do any fancy changing of letters from upper- to lower-case or vice-versa.)

Part C: The Implementation

Files Provided

The starter code contains the `SubsetMeld`, `SlowSubsetMeld`, and `MySubsetMeld` classes, where `SlowSubsetMeld` and `MySubsetMeld` are implementations of the `SubsetMeld` interface. Part C involves you completing the `MySubsetMeld` implementation faster than the provided `SlowSubsetMeld`.

Interface Semantics

`SubsetMeld` represents a set of `n` integer elements {0, 1, 2, …, n-1} grouped into `k` disjoint (non-overlapping) non-empty subsets. For example, {0, 1, 5}, {7}, {2, 4}, {3, 6} would be a `SubsetMeld` structure on the n=8 elements {0, 1, 2, …, 7} grouped into k=4 subsets. It supports the following operations:

`boolean same(int x, int y)` : returns whether elements `x` and `y` are in the same subset. On the example above, `same(0,5)` and `same(7,7)` would return `true`, but `same(0,7)` would return `false`.

`void meld(int x, int y)`: joins the subsets that contain `x` and `y`, i.e. replace the set containing `x` and the set containing `y` with their union. On the example above, `meld(2,6)` would produce the `SubsetMeld` {0, 1, 5}, {7}, {2, 4, 3, 6} (orders within each subset does not matter---they are sets.)

`int size()`: returns the number of underlying elements n (the size of all subsets combined). On the example above, n=8 so `size()` would return 8.

`int numSubsets()`: return the number of subsets k. On the original example, `numSubsets()` would return 4, then after the meld it would return 3.

`SlowSubsetMeld.java` provides a "slow" implementation of the `SubsetMeld` interface. You can and should look at that code if you have questions about the behaviour of the above methods. Some notes about `SlowSubsetMeld`, if you want them:

- It is implemented with integers `n` and `k`, and an array `id` of size `n`, where `id[i]` represents the "subset id" of element `i`. Two elements `x` and `y` are in the same subset if their subset id's match, that is, if `id[x]==id[y]`.
- The constructor takes in an integer `n`, which is the (fixed) size of this particular instance of the SubsetMeld. It initializes each element to be in its own subset.
- The runtime of `meld` and the constructor are O(n) and the other operations are all O(1).

Your Task

[35 marks] Implement `MySubsetMeld` so that `meld` is asymptotically faster than O(n). This may require losing the O(1) runtime for `same`. Easy come, easy go.

This is deliberately vague in its runtime goal. There are many possible approaches that use a variety of concepts from this course. I hope you think of it as a good puzzle; I hope you sit down and think about it before you dive into coding. Many of the best solutions are quite simple, once you figure them out so you don't have to necessarily write a lot of code. It just might take some thinking time (or, maybe not) and some trial and error.

Hint: think about the situations that lead to slow melds in `SlowSubsetMeld` and how you might improve things in that case.

Local Tests

For Part B, you can download some sample input and output files for each question as a zip file (a5-io.zip). Once you get a sense for how to test your code with these input files, write your own tests that test your program more thoroughly (**the provided tests are not exhaustive!**)

For Part C (`MySubsetMeld`), a `Tester` class is provided with some tests of the provided methods. I recommend you add tests to Tester.java to test your `MySubsetMeld` locally (as opposed to on the server).

Testing suggestions:

- Test incrementally, that is, bit-by-bit,
- Test edge cases, e.g. empty files, empty lists, parameters of odd or even sizes,
- Use small test cases at first, then go bigger only when necessary,
- Test individual methods as you write them, even. As in: you can test an incomplete method to make sure it's doing what you're expected thus far.

Tips, Tricks, and FAQs

This section may be updated as the assignment progresses, but for the most up-to-date FAQ, please see the Assignment 5 FAQ post on discord.

How should I approach each problem?

- Make sure you understand it. Construct small examples, and compute (by hand) the expected output. If you aren't sure what the output should be, go no further until you get clarification.
- Now that you understand what you are supposed to output, and you've been able to solve it by hand, think about how you solved it and whether you could explain it to someone. How about explaining it to a computer?
- If it still seems challenging, what about a simpler case? Can you solve a similar or simplified problem? Maybe a special case? If you were allowed to make certain assumptions, could you do it then? Try constructing your code incrementally, solving the smaller or simpler problems, then, only expanding scope once you're sure your simplified problems are solved.

How should I test my code?

- You should be testing your code as you go along.
- Start small so that you can compute the correct solution by hand.
- Edge cases.
- Large inputs. Random inputs. For time.